

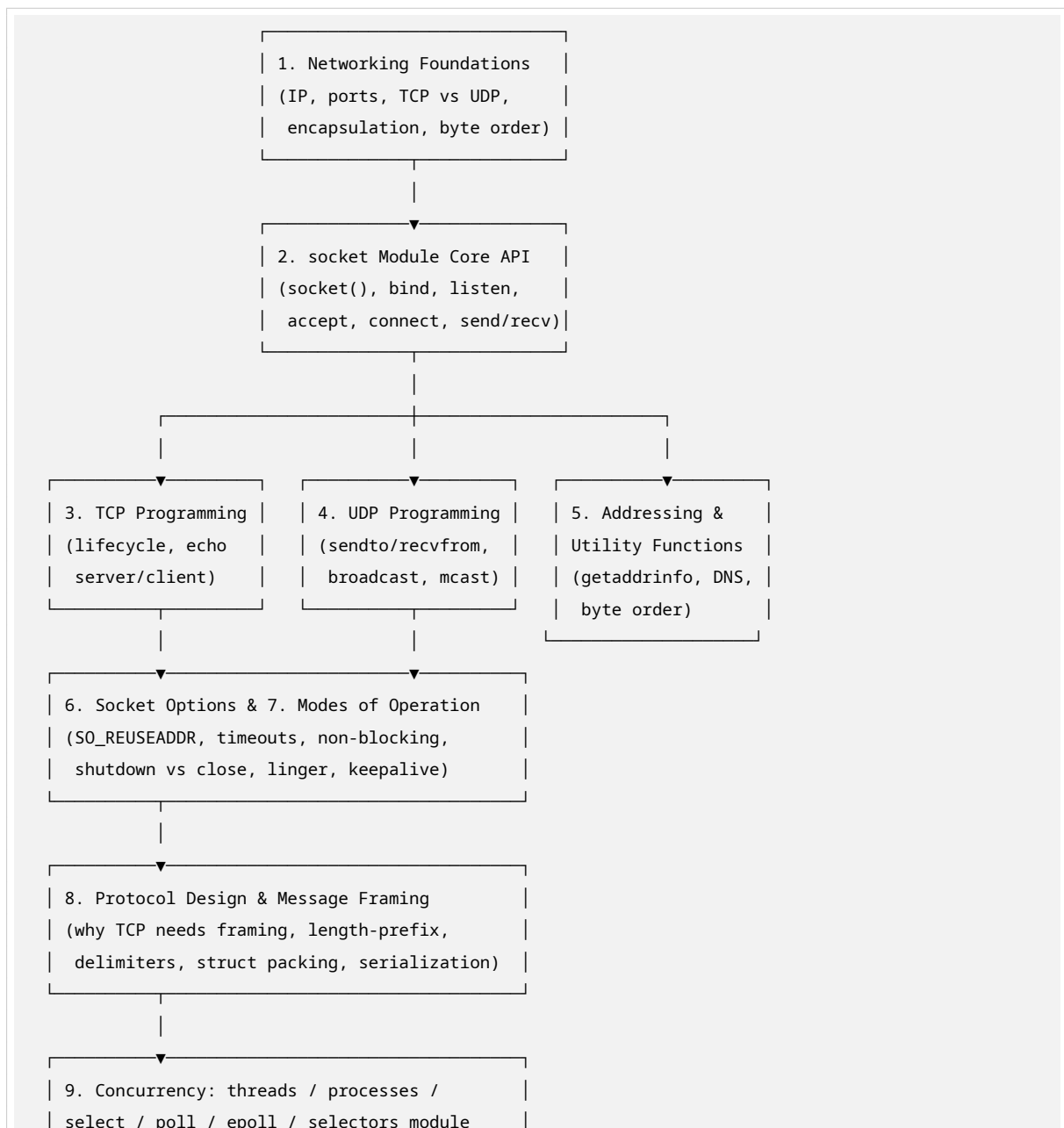
Python Socket Programming — Complete Study Map & Mastery Notes

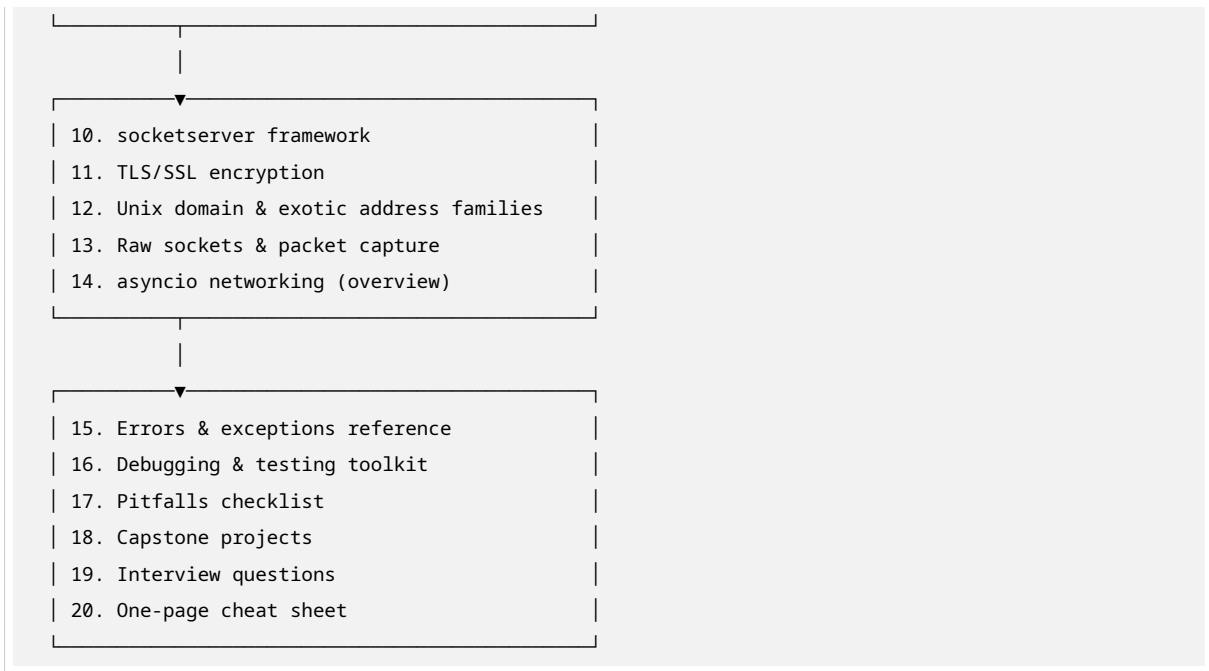
Goal of this document: take you from “what is a socket?” to writing production-grade, concurrent, secure network programs in Python — without skipping the small details that actually bite you in real code and interviews.

How to use: Follow the Study Map (Part 0) in order. Each part has: concepts → minimal working code → the “minor details” section → self-check questions. Do the exercises in Part 18 before claiming mastery.

PART 0 — THE STUDY MAP (ROADMAP)

0.1 The full skill tree





0.2 Phase-based roadmap with time estimates

Phase	Topics	You can...	Est. time
Phase 1	Parts 1–2	Explain what a socket is; create one; name every method	2–3 h
Phase 2	Parts 3–5	Write working TCP/UDP client+server; resolve hosts; pack bytes	4–6 h
Phase 3	Parts 6–8	Control socket behavior; design a protocol with framing	4–6 h
Phase 4	Parts 9–10	Serve 1000s of clients three different ways	6–8 h
Phase 5	Parts 11–14	Add TLS; use Unix sockets; read packets; know asyncio	5–7 h
Phase 6	Parts 15–18	Debug anything; pass the pitfalls checklist; build capstones	6–10 h
Phase 7	Parts 19–20	Self-test for interviews; keep the cheat sheet handy	ongoing

0.3 Mastery checklist (tick only when you can do it *from memory*)

- Write a TCP echo server + client with zero lookups
- Explain why `send()` may transmit fewer bytes than you gave it
- Explain why `recv()` returning `b""` means “peer closed”, not “no data yet”
- Explain what `SO_REUSEADDR` really does (and what it does NOT do)
- Implement length-prefixed message framing
- Explain backlog, SYN queue, and accept queue

- Write a `selectors`-based server handling 10k idle connections
- Explain `shutdown(SHUT_WR)` vs `close()`
- Wrap a socket in TLS, server-side and client-side
- Explain every exception in the `OSError` subtree relevant to sockets
- Explain the difference between TCP “stream” and UDP “datagram” semantics in one sentence
- Describe what `getaddrinfo` returns and why you should iterate over it

PART 1 — NETWORKING FOUNDATIONS (THE NON-NEGOTIABLES)

You cannot debug sockets without this layer. Skim it once, return often.

1.1 What a socket actually is

A **socket** is an operating-system object (a kernel data structure + file descriptor) that represents **one endpoint of a two-way communication channel**. In Python it is surfaced by the `socket` module as an object wrapping that file descriptor.

- A socket is identified by the **5-tuple**: (protocol, local IP, local port, remote IP, remote port).
- TCP server on `10.0.0.5:80` talking to client `203.0.113.7:51514` → 5-tuple (`tcp`, `10.0.0.5`, `80`, `203.0.113.7`, `51514`).
- Because the remote side is part of the identity, one server port can hold **millions** of simultaneous connections — each accepted connection is a *new* socket sharing the same local port.
- In Unix, a socket is a **file descriptor**, so it obeys fd rules: it can leak, it has a per-process limit (`ulimit -n`, default often 1024), it can be passed to child processes, and `select`-family calls watch fds.

Minor detail: `socket.fileno()` returns the integer fd; `-1` after close. `ulimit -n` is the first thing to check when a high-concurrency server dies with `OSError: [Errno 24] Too many open files`.

1.2 The protocol stack you actually use

Application layer	YOUR protocol	HTTP, SMTP, or your own bytes
Transport layer	TCP / UDP	← sockets live HERE
Network layer	IP (v4/v6), ICMP	
Link layer	Ethernet, Wi-Fi	frames, MTU, MAC

Python’s `socket` module is a thin wrapper over the OS **Berkeley sockets API** (POSIX). Everything you learn transfers to C/Go/Java with different syntax.

1.3 IP addresses

IPv4 — 32-bit, dotted-quad `192.168.1.10`, each octet 0–255.

Special ranges you must recognize:

Address	Meaning
127.0.0.0/8 (127.0.0.1 = localhost)	Loopback —never leaves the machine, no NIC involved, no firewall traversal (usually)
0.0.0.0	“All interfaces” when binding (server listens on every NIC); invalid as a destination
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private (RFC 1918) —not routable on the public internet; NAT required
169.254.0.0/16	Link-local (APIPA) —self-assigned when DHCP fails
224.0.0.0/4 (224.0.0.0–239.255.255.255)	Multicast groups
255.255.255.255	Limited broadcast (this LAN segment)

IPv6 — 128-bit, hex groups: `2001:db8::1` (`::` compresses one run of zeros). Loopback is `::1`. “All interfaces” is `::`. Link-local starts `fe80::/10` and needs a **zone/scope id** (`fe80::1%eth0`) because the same prefix exists on every link.

Minor details:

- `socket.AF_INET` `sockaddr` for IPv4 is a **2-tuple** (`host`, `port`); for `AF_INET6` it is a **4-tuple** (`host`, `port`, `flowinfo`, `scope_id`) — almost always (`host`, `port`, `0`, `0`).
- A single dual-stack socket (`AF_INET6` + clearing `IPV6_V6ONLY`) can accept both IPv4 and IPv6 clients; IPv4 clients appear as IPv4-mapped addresses like `::ffff:192.168.1.10`. `socket.create_server()` and `socket.has_dualstack_ipv6()` (Python 3.8+) make this easy.

1.4 Ports

- 16-bit number, 0–65535.
- **0–1023** — well-known/privileged (HTTP 80, HTTPS 443, SSH 22, DNS 53). Binding these needs `root/CAP_NET_BIND_SERVICE` on Linux.
- **1024–49151** — registered (PostgreSQL 5432, Redis 6379...).
- **49152–65535** — ephemeral/dynamic; the OS picks one from here for outgoing client connections automatically.
- **Port 0 when binding** = “OS, pick any free port for me.” Read back the chosen port with `sock.getsockname()[1]`. Essential for tests.

Minor detail: a port is per-protocol. TCP port 53 and UDP port 53 are different namespaces.

1.5 TCP vs UDP — the decision table

Property	TCP (<code>SOCK_STREAM</code>)	UDP (<code>SOCK_DGRAM</code>)
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Guaranteed delivery, retransmission	No guarantees —loss, duplication, reorder all possible
Ordering	In-order byte stream	None

Table 3 – continued

Property	TCP (SOCK_STREAM)	UDP (SOCK_DGRAM)
Data model	Stream —no message boundaries	Datagrams —boundaries preserved
Flow/congestion control	Yes (sliding window, AIMD)	No
Overhead	~20-byte header + state	8-byte header, stateless
Use when	Correctness matters (files, HTTP, DB)	Latency > reliability (DNS, VoIP, games, QUIC*)

* Modern protocols (QUIC/HTTP3) build reliability *on top of* UDP in userspace.

The one-sentence mastery line: TCP gives you a reliable, ordered **pipe of bytes** (like a file); UDP gives you **individual envelopes** that may arrive late, twice, out of order, or never.

Minor details:

- TCP “connection” is just synchronized state on both ends; there is no physical circuit. It survives routing changes and only dies on timeout/RST/close.
- UDP `connect()` exists! It does **not** handshake — it just sets a default destination and filters incoming datagrams to that peer, enabling `send()/recv()` and async error delivery (ICMP port unreachable → `ConnectionRefusedError` on next call, Linux).
- One UDP `sendto` = one datagram = one `recvfrom` on the other side (or nothing). One TCP `send` may be split or merged arbitrarily with other sends — see Part 8.

1.6 Key transport mechanics (interview favorites)

TCP 3-way handshake: SYN → SYN+ACK → ACK. Happens inside `connect()/accept()`; you never see it in Python.

TCP teardown: FIN → ACK, then the other direction’s FIN → ACK (4 segments). The side that closes first enters **TIME_WAIT** for $2 \times \text{MSL}$ (typically 60s on Linux) — this is why rebinding a restarted server fails without `SO_REUSEADDR`.

MSS/MTU: path MTU typically 1500 bytes; TCP MSS 1460. Sending 1 MB is split into ~700 segments invisibly.

Nagle’s algorithm: TCP buffers small writes to send fuller segments → up to ~40–200 ms added latency for chatty protocols. Disable per-socket with `TCP_NODELAY` (Part 6). This is a classic “why is my RPC slow?” answer.

UDP max payload: theoretical 65507 bytes over IPv4 ($65535 - 20 \text{ IP} - 8 \text{ UDP}$). Practically keep datagrams ~1472 bytes to avoid IP fragmentation, which tanks reliability (lose one fragment, lose all).

1.7 Byte order (endianness)

- Multi-byte integers have two serializations: **big-endian** (most significant byte first) and **little-endian**.
- x86/ARM are little-endian; **network byte order is big-endian**.
- Consequences: port 80 = `0x0050`. If you memcopy a little-endian `uint16` into a packet you get byte-swapped garbage on the wire.

- Python helpers: `socket.htons()`, `socket.htonl()`, `socket.ntohs()`, `socket.ntohl()` (host network short/long).
- In practice you use `struct` with `!` (network) prefix: `struct.pack("!IH", msg_id, flags)` — covered in Part 8.

PART 2 — THE `socket` MODULE CORE API

2.1 Creating a socket

```
import socket

s = socket.socket(family=socket.AF_INET,      # address family
                  type=socket.SOCK_STREAM,    # socket type
                  proto=0)                    # protocol (0 = default for type)
```

Address families (family):

Constant	Meaning	sockaddr shape
AF_INET	IPv4	(host: str, port: int)
AF_INET6	IPv6	(host, port, flowinfo, scope_id)
AF_UNIX / AF_LOCAL	Unix domain (same-machine IPC)	path string, or <code>\0abstract</code> (Linux)
AF_PACKET	Linux raw link-layer frames	(ifname, proto[, pkttype...])
AF_BLUETOOTH, AF_CAN, AF_NETLINK, ...	Exotic, platform-specific	varies

Socket types (type):

Constant	Meaning
SOCK_STREAM	TCP —reliable ordered byte stream
SOCK_DGRAM	UDP —datagrams
SOCK_RAW	Raw IP packets (needs root; you build headers)
SOCK_SEQPACKET	Reliable ordered datagrams (Unix domain, Bluetooth) —rare
SOCK_RDM	Reliably-delivered messages —very rare

Type flags (Linux, OR-able): `SOCK_NONBLOCK` (create non-blocking) and `SOCK_CLOEXEC` (close on `exec`) — Python sets `CLOEXEC` by default since 3.4 (PEP 446).

proto: almost always `0` (let OS pick: TCP for stream, UDP for datagram). Raw sockets use e.g. `socket.IPPROTO_ICMP`.

Minor details:

- Sockets are **context managers**: with `socket.socket()` as `s`: closes on exit. Always use this in examples and tests.

- `socket.socketpair()` creates two connected Unix sockets — great for testing and for waking up a selector loop from another thread.
- `socket.fromfd(fd, family, type, proto)` wraps an existing fd (duplicates it).

2.2 The complete method map

For servers (TCP listening sockets)

Method	Purpose
<code>bind(address)</code>	Assign local IP+port
<code>listen([backlog])</code>	Mark passive; set accept-queue size
<code>accept()</code> → (conn, addr)	Dequeue one pending connection; conn is a new socket

For clients

Method	Purpose
<code>connect(address)</code>	Blocking connect; raises on failure
<code>connect_ex(address)</code>	Same but returns <code>errno</code> int instead of raising (0 = success)

Data transfer

Method	Purpose
<code>send(bytes)</code> → int	Send up to <code>len(bytes)</code> ; returns count actually sent
<code>sendall(bytes)</code>	Loop <code>send</code> until everything is out (returns None, raises on error)
<code>sendmsg(buffers[, ancdata[, flags]])</code>	Scatter/gather + ancillary data (fd passing) — advanced
<code>sendfile(file[, offset[, count]])</code>	Zero-copy file send (Unix; wraps <code>os.sendfile</code>)
<code>recv(bufsize[, flags])</code> → bytes	Receive up to <code>bufsize</code> ; <code>b""</code> = orderly peer close
<code>recv_into(buffer[, nbytes])</code> → int	Receive into a preallocated buffer/memoryview
<code>recvmsg(bufsize[, ancbufsize])</code>	+ ancillary data
<code>sendto(bytes, address) / recvfrom(bufsize)</code> → (data, addr)	UDP I/O (no connection)
<code>sendto/recvfrom</code> on connected UDP	Also legal after <code>connect()</code> ; plain <code>send/recv</code> too

Lifecycle & state

Method	Purpose
<code>shutdown(how)</code>	Half/Full close: <code>SHUT_RD</code> , <code>SHUT_WR</code> , <code>SHUT_RDWR</code> — sends FIN but keeps fd

Table 9 – continued

Method	Purpose
<code>close()</code>	Release the fd (decrements refcount; underlying socket dies when last fd closes)
<code>detach() → fd</code>	Hand the fd out without closing it
<code>dup()</code>	Duplicate the socket object
<code>setblocking(bool) / settimeout(sec) / gettimeout()</code>	I/O mode control
<code>set_inheritable(bool) / get_inheritable()</code>	fd inheritance across fork/exec

Introspection

Method	Purpose
<code>getsockname()</code>	Local address (use after <code>bind("", 0)</code> to learn the port)
<code>getpeername()</code>	Remote address (raises if not connected)
<code>getsockopt(level, optname[, buflen])</code>	Read a socket option
<code>setsockopt(level, optname, value)</code>	Set a socket option
<code>ioctl(control, option)</code>	Platform-specific (Windows <code>SIO_RCVALL</code> , <code>SIO_KEEPA_LIVE_VALS</code>)
<code>gettimeout(), fileno()</code>	Metadata

Module-level conveniences (use these first!)

Function	Purpose
<code>socket.create_connection(address, timeout=..., source_address=...)</code>	Does <code>getaddrinfo</code> + try-each + connect for you. The right way to make a client.
<code>socket.create_server(address, *, family=AF_INET, backlog=None, reuse_port=False, dualstack_ipv6=False)</code>	(3.8+) <code>bind</code> + <code>listen</code> in one call, <code>SO_REUSEADDR</code> handled properly per-platform. The right way to make a server socket.
<code>socket.getaddrinfo(host, port, family=0, type=0, proto=0, flags=0)</code>	Resolve to a list of 5-tuples (family, type, proto, canonname, sockaddr)
<code>socket.gethostbyname(), getfqdn(), gethostbyname(), gethostbyname_ex(), gethostbyaddr()</code>	Legacy IPv4-only DNS helpers
<code>socket.inet_aton() / inet_ntoa()</code>	IPv4 str → packed 4 bytes
<code>socket.inet_pton() / inet_ntop()</code>	v4 and v6 str → packed (family arg required)
<code>socket.htons/htonl/ntohs/ntohl</code>	Byte-order conversion
<code>socket.getservbyname("http") → 80 / getservbyport(80) → "http"</code>	/etc/services lookups
<code>socket.getprotobyname("tcp")</code>	/etc/protocols
<code>socket.if_nameindex(), if_nameindex(), if_indextoname()</code>	NIC enumeration
<code>socket.has_ipv6, socket.has_dualstack_ipv6()</code>	Capability probes

Minor details:

- `create_connection` tries each `getaddrinfo` result **sequentially** (no Happy Eyeballs parallelism); each gets the full timeout. For multi-homed hosts, set a sane timeout or the worst case is `timeout × #addresses`.
- `accept()` on a socket with a timeout raises `socket.timeout` if nothing arrives in time — handy for graceful shutdown loops.
- Every method that does I/O can raise `InterruptedError` (signal arrived); since Python 3.5, syscalls are **automatically retried** after signal handlers run (PEP 475) — you rarely need manual retry loops.

2.3 Minimal TCP client/server (the “hello world” to memorize)

Server (`echo_server.py`):

```
import socket

HOST, PORT = "127.0.0.1", 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) # restart-friendly
    s.bind((HOST, PORT))
    s.listen(128) # backlog
    print("Listening on", s.getsockname())
    conn, addr = s.accept() # blocks until a client connects
    with conn: # conn is a NEW socket, dedicated to this client
        print("Connected by", addr)
        while True:
            data = conn.recv(1024) # blocks; b"" means client closed
            if not data:
                break
            conn.sendall(data) # echo back
print("Server exiting")
```

Client:

```
import socket

with socket.create_connection(("127.0.0.1", 65432), timeout=5) as s:
    s.sendall(b"hello, sockets")
    print("echo:", s.recv(1024)) # b'hello, sockets'
```

Test without code: `nc 127.0.0.1 65432` (netcat) or `telnet 127.0.0.1 65432`.

PART 3 — TCP PROGRAMMING IN DEPTH

3.1 The TCP server lifecycle (memorize this diagram)

```
socket() → bind() → listen() → accept() → recv/send loop → close(conn)
|           |           |           |
|           |           |           └─ repeat for each client ─┘
```

- | | | socket becomes PASSIVE (listen-only);
- | | | it NEVER carries data itself
- | | | fails with EADDRINUSE if port busy/TIME_WAIT
- | | | (fix: SO_REUSEADDR)
- | | | optionally setsockopt(SO_REUSEADDR) BEFORE bind

Critical conceptual point: the **listening socket** only accepts; each `accept()` returns a **brand-new connected socket**. The listening socket and the connection sockets are independent fds — closing one doesn't close the others.

3.2 bind()

- Binds to a (host, port).
- host="" or "0.0.0.0" → all IPv4 interfaces. ":::" → all IPv6. "127.0.0.1" → loopback only (invisible to the LAN — do this for anything security-sensitive in dev).
- port=0 → ephemeral auto-assign; read back via `getsockname()`.
- Errors: EADDRINUSE (port taken/TIME_WAIT), EADDRNOTAVAIL (IP not local), EACCES (privileged port without root).

Minor detail: on **Windows**, `SO_REUSEADDR` has *different, dangerous* semantics (allows hijacking a bound port). Python's `socket.create_server()` handles this correctly per-platform (it does NOT set `SO_REUSEADDR` on Windows; `SO_EXCLUSIVEADDRUSE` exists for that). Hand-rolled cross-platform servers should use `create_server`.

3.3 listen(backlog) — the queue everyone gets wrong

- Converts the socket to passive mode. **No data ever flows on it.**
- **backlog** = max number of **queued** (not-yet-accepted) connections.
- The kernel actually keeps **two queues** (Linux):
 1. **SYN queue** — half-open connections (SYN received, handshake incomplete). Sized via `tcp_max_syn_backlog/syncookies`.
 2. **Accept queue** — fully handshake connections waiting for `accept()`. Length = `min(backlog, /proc/sys/net/core/somaxconn)` (default 4096 on modern Linux, 128 historically).
- If the accept queue is full, new connections get dropped or RST (client sees `ECONNREFUSED` or hangs).
- **Python 3.5+**: if you omit `backlog`, it defaults to `SOMAXCONN` (a sane big value). Still, pass an explicit value in servers you care about.
- **Key insight:** a large backlog = concurrency. The queue only buys time while your `accept()` loop is busy. Real concurrency comes from Part 9.

3.4 accept()

```
conn, addr = listen_sock.accept()
# conn: new socket, blocking by default, inherits... see below
# addr: client's (ip, port) for AF_INET
```

Minor details:

- Returned socket is blocking **even if the listening socket is non-blocking** — on Linux, unless you created it with `SOCK_NONBLOCK...` actually, POSIX says the new socket does *not* inherit `O_NONBLOCK`

from the listener; Linux's `accept4()` can. **In Python: always `conn.setblocking(...)` explicitly** — never assume.

- `addr` for IPv6 is the 4-tuple; for `AF_UNIX` it's the peer path (often empty).
- If the client disconnects between handshake and your `accept()`, the first `recv` may raise `ConnectionResetError` — be ready.

3.5 `connect()` / `connect_ex()`

- Blocking until handshake completes or fails. Failure modes: `ConnectionRefusedError` (RST — nothing listening), `TimeoutError` (SYN dropped by firewall), `OSError: No route to host` (ICMP unreachable), `gaierror` (DNS failure).
- The OS auto-binds an ephemeral local port; override with `s.bind((local_ip, 0))` before `connect`, or `source_address` in `create_connection`.
- **Non-blocking connect pattern** (needed for async/connect-timeout logic):

```
import socket, selectors

s = socket.socket()
s.setblocking(False)
err = s.connect_ex((host, port))          # returns immediately
# err is 0, EINPROGRESS, EWOULDBLOCK, or EALREADY — all "in progress" on non-blocking
sel = selectors.DefaultSelector()
sel.register(s, selectors.EVENT_WRITE)
events = sel.select(timeout=5)            # writable = connect finished
if events:
    so_error = s.getsockopt(socket.SOL_SOCKET, socket.SO_ERROR) # 0 = success
    if so_error == 0:
        print("connected!")
    else:
        raise OSError(so_error, "connect failed")
else:
    raise TimeoutError("connect timed out")
```

`SO_ERROR` retrieval after writable-notification is **the** canonical way to learn non-blocking connect results. Interviewers love it.

3.6 `send()` vs `sendall()` — the #1 TCP bug

- `send(data)` returns the number of bytes **actually handed to the kernel**, which may be $< \text{len}(\text{data})$ (kernel send buffer full). With blocking sockets this is rare for small payloads but guaranteed to happen eventually at scale or with non-blocking I/O.
- `sendall(data)` loops until all bytes are sent; returns `None`; raises on error. **Use `sendall` unless you have a reason not to.**
- Neither guarantees the peer *received* anything — only that the local kernel accepted it. TCP reliability is asynchronous; the failure may surface on a *later* call as `BrokenPipeError`/`ConnectionResetError`.

Minor details:

- `sendall` with a timeout: the timeout applies to **each underlying syscall**, not the whole operation

— a slow trickle can make `sendall` run far longer than your timeout. For a hard deadline, implement your own loop with `time.monotonic()`.

- On blocking sockets, Python ignores `SIGPIPE` and raises `BrokenPipeError` instead of killing your process (unlike C).
- `send(b'')` sends nothing, returns 0. It is NOT a close signal. (UDP: sends a zero-length datagram.)

3.7 `recv()` — reading the stream correctly

```
data = sock.recv(4096)
```

- Returns **up to** 4096 bytes — possibly 1 byte. TCP is a stream; the kernel gives you whatever is available.
- Returns `b''` **only** when the peer has performed an orderly shutdown (`close/shutdown(SHUT_WR)`) and all prior data was consumed. This is your “end of stream” signal.
- Never returns `b''` for “no data right now” — blocking sockets just block; non-blocking raise `BlockingIOError`; timeout sockets raise `socket.timeout`.
- **Buffer size:** powers of two, 4096–65536. Bigger = fewer syscalls; 16 KiB is a good default. There’s no prize for matching the peer’s send size — sizes are irrelevant to semantics.
- `recv_into(memoryview(bytearray(n)))` avoids copies in hot paths.
- Flags: `socket.MSG_PEEK` (look without consuming), `socket.MSG_WAITALL` (block until full or EOF/error), `socket.MSG_DONTWAIT` (this call only, non-blocking). Platform support varies; test.

Read-N-bytes helper — paste into every project:

```
def recv_exact(sock, n: int) -> bytes:
    """Read exactly n bytes or raise. TCP streams don't respect your boundaries."""
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError(f"peer closed after {len(buf)}/{n} bytes")
        buf.extend(chunk)
    return bytes(buf)
```

3.8 `shutdown()` vs `close()` — half-closing

```
sock.shutdown(socket.SHUT_WR)  # "I'm done sending" → sends FIN; peer's recv() → b''
# ...but you can still READ the peer's response. Then:
sock.close()
```

- `SHUT_RD` — no more reads (pending data discarded, peer gets nothing special).
- `SHUT_WR` — sends FIN; the classic **half-close**. Peer sees EOF but may keep sending. Use cases: client sends request then `SHUT_WR` to say “request complete, now talk”; proxying.
- `SHUT_RDWR` — both.
- **Crucial difference:** `shutdown` operates on the *underlying connection* immediately, regardless of how many fds/duplicates exist. `close` just drops your fd; the connection lives until the *last* fd (incl. forked children, dups) closes. A forked child holding the socket can keep a connection alive against your will.
- After `close()`, any use raises `OSError: [Errno 9] Bad file descriptor`.

- Always shutdown before close on connected sockets when the peer needs a clean EOF — especially with protocols that read-until-EOF.

3.9 makefile() — treating a socket like a file

```
f = sock.makefile("r", encoding="utf-8", newline="\n")
for line in f:           # line-delimited protocols become trivial
    handle(line)
w = sock.makefile("w")
w.write("OK\n"); w.flush()
```

Minor details:

- The file object has its **own buffer**. Mixing `f.read()` and `sock.recv()` loses data (some bytes sit in the file buffer). Pick one abstraction per direction.
- Closing all file objects **does not close the socket** until you also `sock.close()` (since 3.6 the socket closes only when both are closed — but shutdown still hits the connection instantly).
- With **timeouts**, `makefile` reads are unreliable — an internal buffer read can raise timeout even though data partially arrived, and the partial data is lost. Docs explicitly warn against timeout + `makefile` for reads.

3.10 A robust threaded TCP server (Phase-2 baseline)

```
import socket
import threading

def handle_client(conn: socket.socket, addr):
    with conn:
        try:
            while True:
                data = conn.recv(4096)
                if not data:
                    break
                conn.sendall(data)
        except (ConnectionResetError, BrokenPipeError):
            pass          # client vanished mid-conversation
        finally:
            print("closed:", addr)

def serve(host="127.0.0.1", port=65432):
    with socket.create_server((host, port)) as srv:  # bind+listen+REUSEADDR done right
        print("listening:", srv.getsockname())
        while True:
            conn, addr = srv.accept()
            threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()

if __name__ == "__main__":
    serve()
```

daemon=True so stray client threads don't block interpreter exit. Real services also track threads and shutdown() their sockets during graceful stop — see Part 9.

PART 4 — UDP PROGRAMMING IN DEPTH

4.1 The UDP pattern (no listen, no accept)

```
# ---- server ----
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.bind(("127.0.0.1", 65432))
    print("UDP server on", s.getsockname())
    while True:
        data, addr = s.recvfrom(65535)      # one datagram, with sender address
        print(f"{addr} says {data!r}")
        s.sendto(data.upper(), addr)        # reply addressed explicitly

# ---- client ----
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.settimeout(2)                         # UDP = you MUST handle loss yourself
    s.sendto(b"ping", ("127.0.0.1", 65432))
    try:
        data, server = s.recvfrom(65535)
        print("got", data, "from", server)
    except socket.timeout:
        print("no reply (lost?) — retry per your protocol")
```

Everything differs from TCP here:

- No listen()/accept(). One socket talks to **everyone**.
- Every read gives you a **complete datagram** plus the sender's address (recvfrom).
- Every write needs a destination (sendto), unless you connect()ed.
- recvfrom(bufsize) with a datagram larger than bufsize → the datagram is **truncated**, the rest discarded (some platforms signal this via MSG_TRUNC). Use 65535 to be safe.

4.2 Connected UDP

```
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.connect(("8.8.8.8", 53))      # NO handshake! just sets default peer + filter
s.send(b"\x00\x00...")         # send/recv now work
```

Effects of UDP connect():

1. Default destination for send()/sendall().
2. Kernel **drops datagrams from anyone else**.

3. (Linux) ICMP errors become socket errors — sending to a closed port then reading raises `ConnectionRefusedError`.

Classic trick: discover which local IP would reach a target —

```
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.connect(("8.8.8.8", 80))
    print(s.getsockname()[0])    # e.g. '192.168.1.23' — no packet is actually sent
```

4.3 Broadcast (LAN-wide, one-to-all)

```
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)    # REQUIRED or PermissionError
    s.sendto(b"hello LAN", ("255.255.255.255", 5000))
    # better: directed broadcast, e.g. ("192.168.1.255", 5000)
```

- Broadcast is IPv4-only (IPv6 replaced it with multicast).
- Receivers just bind the port; no special option needed to *receive*.
- Most routers do **not** forward broadcasts — it stays on the local segment.

4.4 Multicast (one-to-group)

```
# ---- sender ----
import socket, struct
MCAST_GRP, MCAST_PORT = "224.1.1.1", 5007

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 2)    # how far packets may travel
sock.sendto(b"news flash", (MCAST_GRP, MCAST_PORT))

# ---- receiver ----
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)    # multiple receivers on one host
sock.bind(("", MCAST_PORT))
mreq = struct.pack("4s4s", socket.inet_aton(MCAST_GRP), socket.inet_aton("0.0.0.0"))
sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
while True:
    print(sock.recvfrom(1024))
# leave: sock.setsockopt(socket.IPPROTO_IP, socket.IP_DROP_MEMBERSHIP, mreq)
```

Minor details:

- TTL semantics for multicast differ from unicast: 0=host, 1=subnet (default), <32=site, <255=global (administrative scoping conventions).
- Group range 224.0.0.0/4; 224.0.0.x is reserved for link-local protocols — pick 239.x (administratively scoped) for private apps.
- On multi-homed hosts, the second `inet_aton` arg should be the joining interface's IP, not 0.0.0.0.

4.5 Building reliability on UDP (what interviews ask)

If you choose UDP, you inherit its problems. Your protocol must add (as needed):

- **Sequence numbers** → detect loss/reorder/duplicates.
- **ACKs + retransmission with timeout** → reliability.
- **Checksums** beyond UDP's own (which is optional in IPv4!) → integrity.
- **Session/connection IDs** → demultiplexing.
- **Congestion control** → be a good internet citizen (this is the hard one; look at QUIC/LEDBAT).

Examples in the wild: DNS (tiny + retry), RTP (media, tolerate loss), QUIC (full reliable transport in userspace), game state sync (latest-value-wins → loss is fine).

Minor detail: UDP's IPv4 header checksum is optional and often 0; Ethernet FCS protects frames, but if you tunnel/forward, corruption can survive. For critical payloads add an app-level checksum or AEAD (e.g., DTLS).

PART 5 — ADDRESSING, DNS & UTILITY FUNCTIONS

5.1 `getaddrinfo` — the only resolver you should use

```
import socket
infos = socket.getaddrinfo("example.com", 443, socket.AF_UNSPEC, socket.SOCK_STREAM)
for family, type_, proto, canonname, sockaddr in infos:
    print(family, type_, proto, canonname, sockaddr)
# (AddressFamily.AF_INET, SOCK_STREAM, 6, '', ('93.184.216.34', 443))
# (AddressFamily.AF_INET6, SOCK_STREAM, 6, '', ('2606:2800:220:1:... ', 443, 0, 0))
```

- Returns **all** matching endpoints (IPv4+IPv6), already in connect-ready `sockaddr` form. Iterate and try each — exactly what `create_connection` does.
- flags: `AI_PASSIVE` (for bind — fills wildcard when host is None), `AI_CANONNAME`, `AI_NUMERICHOST` (skip DNS; string must already be an IP), `AI_ADDRCONFIG`.
- `proto 6 = TCP`, `17 = UDP` (from `getprotobyname`).
- Raises `socket.gaierror` (subclass of `OSError`) on resolution failure — e.g. `[Errno -2] Name or service not known`.

Hand-rolled robust client (what `create_connection` saves you from writing):

```
import socket

def connect_any(host, port, timeout=5):
    last_err = None
    for af, st, proto, _, sa in socket.getaddrinfo(host, port, 0, socket.SOCK_STREAM):
        s = socket.socket(af, st, proto)
        s.settimeout(timeout)
        try:
            s.connect(sa)
            return s
        except OSError as e:
            last_err = e
    s.close()
```



```
raise last_err or OSError("no address resolved")
```

5.2 Legacy DNS helpers (IPv4-only, but everywhere in old code)

Function	Returns	Notes
<code>socket.gethostname()</code>	this machine's name	e.g. 'my-laptop'
<code>socket.getfqdn()</code>	fully-qualified name	may just repeat hostname
<code>socket.gethostbyname("host")</code>	one IPv4 string	no v6, raises <code>gaierror</code>
<code>socket.gethostbyname_ex("host")</code>	(hostname, aliaslist, ipaddrlist)	
<code>socket.gethostbyaddr("1.2.3.4")</code>	reverse DNS (name, aliases, addrs)	raises <code>herror</code> if no PTR record
<code>socket.gethostbyname(socket.gethostbyname_ex("name")[2])</code>	name and IP	unreliable —often returns 127.0.1.1; use the UDP-connect trick (§4.2) instead

Minor detail: `herror` (host resolution) vs `gaierror` (`addrinfo`) — both subclass `OSError`; catch `OSError` to get both.

5.3 Address packing utilities

```
socket.inet_aton("192.168.1.1")      # b'\xc0\xa8\x01\x01' (IPv4 only)
socket.inet_ntoa(b'\xc0\xa8\x01\x01') # '192.168.1.1'
socket.inet_pton(socket.AF_INET6, "::1")      # 16 bytes
socket.inet_ntop(socket.AF_INET, b'\x7f\x00\x00\x01') # '127.0.0.1'
```

- Use `inet_pton/inet_ntop` — they handle both families.
- Packed form is what goes into `struct.pack` for headers, multicast structs, netmasks.
- Validating an IP string without DNS: try: `socket.inet_pton(family, s)` — with `AI_NUMERICHOST` in `getaddrinfo` as the two-family alternative.

PART 6 — SOCKET OPTIONS: `setsockopt` / `getsockopt`

```
s.setsockopt(level, optname, value)
v = s.getsockopt(level, optname)      # returns int
buf = s.getsockopt(level, optname, 64) # returns bytes (for struct-valued options)
```

`level` is usually `socket.SOL_SOCKET` (generic), `socket.IPPROTO_TCP`, `socket.IPPROTO_IP`, or `socket.IPPROTO_IPV6`.

6.1 The essential options table

Option (level)	Type	What it does	When to use
SO_REUSEADDR (SOL_SOCKET)	int bool	Allow bind despite TIME_WAIT sockets (Unix) / port hijack (Windows —beware!)	Every TCP server on Unix
SO_REUSEPORT (SOL_SOCKET)	int bool	Multiple sockets bind the same port; kernel load-balances accept among them	Multi-process servers (Linux 3.9+)
SO_KEEPALIVE (SOL_SOCKET)	int bool	Kernel sends TCP keepalive probes on idle connections	Detect dead peers on long-idle conns
TCP_NODELAY (IPPROTO_TCP)	int bool	Disable Nagle → send small packets immediately	RPC/games/any latency-sensitive request-reply
SO_RCVBUF / SO_SNDBUF (SOL_SOCKET)	int bytes	Kernel buffer sizes (Linux doubles what you set for bookkeeping; caps in /proc/sys/net/core/rmem_max)	Throughput tuning
SO_RCVTIMEO / SO_SNDTIMEO (SOL_SOCKET)	struct.pack('ll', sec, usec)	Kernel-level I/O timeouts (alternative to settimeout)	Rarely needed; settimeout is easier
SO_LINGER (SOL_SOCKET)	struct.pack('ii', onoff, seconds)	close() behavior: (1, N) = wait up to N s for unsent data; (1, 0) = send RST, abort immediately	Force-reset a connection; avoid TIME_WAIT (dangerous)
SO_BROADCAST (SOL_SOCKET)	int bool	Permission to send to broadcast addresses	UDP broadcast (required!)
SO_ERROR (SOL_SOCKET)	read-only int	Pending async error (clears after read)	Non-blocking connect result
SO_TYPE (SOL_SOCKET)	read-only int	SOCK_STREAM/SOCK_DGRAM of an unknown socket	Introspection
SO_ACCEPTCONN (SOL_SOCKET)	read-only int	Is this a listening socket?	Introspection
SO_BINDTODEVICE (SOL_SOCKET)	bytes iface name	Pin socket to a NIC (needs root)	Multi-homed routing control
SO_INCOMING_CPU, SO_BUSY_POLL...	—	Linux perf esoterica	Know they exist
TCP_KEEPIDLE / TCP_KEEPINTVL / TCP_KEEPCNT (IPPROTO_TCP)	int	Tune keepalive: start after N s idle, probe every M s, give up after K fails (Linux)	Long-lived conns (DB pools, SSH-like)
TCP_KEEPALIVE (IPPROTO_TCP)	int	macOS/Windows equivalent of KEEPIDLE	Cross-platform keepalive tuning

Table 13 – continued

Option (level)	Type	What it does	When to use
IP_MULTICAST_TTL, IP_MULTICAST_LOOP, IP_ADD_MEMBERSHIP (IPPROTO_IP)	—	Multicast control (§4.4)	UDP multicast
IPV6_V6ONLY (IPPROTO_IPV6)	int bool	Disable dual-stack on a v6 socket	Explicit v6-only services
TCP_MAXSEG, TCP_CORK, TCP_QUICKACK (IPPROTO_TCP)	int	Linux TCP micro-tuning	Rare; know TCP_CORK pairs with sendfile batching

6.2 SO_REUSEADDR — precise semantics (interview gold)

- **What it does (Unix):** lets `bind()` succeed when sockets in `TIME_WAIT` state still occupy the port. Without it, restarting a server that had clients → `OSError: [Errno 98] Address already in use` for up to ~60 s.
- **What it does NOT do:** let you bind a port with an active `LISTEN` socket on it (that’s `SO_REUSEPORT`).
- **When to set:** before `bind()`, on the server socket, always (Unix).
- **Windows difference:** there it *allows two active sockets on the same port* (security hazard) — which is why `socket.create_server()` doesn’t set it on Windows, and why Windows has `SO_EXCLUSIVEADDRUSE` for the “safe restart” semantics.

6.3 Keepalive — detecting silently dead peers

TCP has **no traffic on idle connections**. If a client vanishes (Wi-Fi drop, power loss) without `FIN/RST`, your server’s `recv()` will block **forever**. Fixes:

1. **App-level heartbeats** (best — you control the timeout).
2. **TCP keepalive:**

```
s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDLE, 60) # start probing after 60s idle
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDLE, 10) # probe every 10s
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT, 5) # declare dead after 5 misses
# → after ~110s of silence, recv() raises/returns EOF
```

Defaults without tuning: probes start after **7200 s** (2 h) — practically useless unless tuned.

6.4 SO_LINGER and the RST trick

```
import struct
s.setsockopt(socket.SOL_SOCKET, socket.SO_LINGER, struct.pack("ii", 1, 0))
s.close() # → sends RST instead of FIN; unsent data discarded; no TIME_WAIT
```

Uses: abortive disconnects in protocols, tests. Dangers: peer sees `ConnectionResetError` and may lose in-flight data. Do **not** use it to “avoid `TIME_WAIT`” in production — `TIME_WAIT` exists to protect you from segment mixups between incarnations of a connection.

6.5 Buffer sizes

```
s.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 262144)
print(s.getsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF)) # Linux: 524288 (doubled!)
```

- Bigger buffers → deeper TCP window → better throughput on high-latency links (bandwidth-delay product rule of thumb: $\text{buffer} \geq \text{bandwidth} \times \text{RTT}$).
- Kernel caps: `/proc/sys/net/core/rmem_max`, `wmem_max`. Autotuning (`tcp_moderate_rcvbuf`) often makes manual setting unnecessary — measure first.

PART 7 — MODES OF OPERATION: BLOCKING, TIMEOUT, NON-BLOCKING

Every socket is in exactly one of three modes:

Mode	Set by	Behavior when operation can't complete
Blocking (default)	<code>s.setblocking(True)</code> / <code>s.settimeout(None)</code>	Waits indefinitely
Timeout	<code>s.settimeout(5.0)</code>	Waits up to N s, then raises <code>socket.timeout</code> (alias of <code>TimeoutError</code> since 3.10)
Non-blocking	<code>s.setblocking(False)</code> / <code>s.settimeout(0.0)</code>	Returns/raises immediately: <code>BlockingIOError</code> (EAGAIN/EWOULDBLOCK)

```
s.gettimeout() # None → blocking; float → timeout; 0.0 → non-blocking
```

Minor details:

- The mode is per-socket and affects **all** its I/O including `connect` and `accept`.
- `socket.timeout` is raised as `TimeoutError` — catch both for older code: `except (socket.timeout, TimeoutError)`.
- A timeout mid-operation can leave state ambiguous: a `sendall` may have sent part of the data before timing out. After a timeout on a stream socket, the safest recovery is usually `close` + `reconnect`.
- Timeouts don't compose with `makefile` reads (§3.9) and don't bound multi-call operations.
- Non-blocking mode is the foundation of event loops (Part 9). Direct use means loops of `try: recv except BlockingIOError: sleep` — never do this in production; use `selectors`.

PART 8 — PROTOCOL DESIGN & MESSAGE FRAMING (the heart of real socket work)

8.1 Why TCP *must* have framing

TCP delivers a **byte stream with no message boundaries**. If the client does:

```
s.sendall(b"HELLO")
s.sendall(b"WORLD")
```

the server's `recv()` might return: `b"HELLOWORLD"`, or `b"HE"` then `b"LL0"` then `b"WORLD"`, or any other split/merge. **Any code that assumes one send = one recv is broken.** It may pass on localhost (loopback is fast and buffers are big) and explode in production.

The four classic framing strategies:

Strategy	Idea	Pros	Cons
Fixed-length	Every message exactly N bytes	Trivial	Wasteful, inflexible
Delimiter	Read until a marker (<code>\n</code> , <code>\0</code>)	Simple, human-debuggable	Must escape delimiter inside payload; unbounded line = DoS vector
Length-prefix	Header carries payload length	General, binary-safe, bounded	Header itself needs framing (it's fixed-length)
Self-describing	Protocol grammar (HTTP chunked, TLS records)	Powerful	Complex

Length-prefixing is the default answer for custom binary protocols.

8.2 struct — packing the header

```
import struct

# ! = network byte order (big-endian). I = uint32, H = uint16, B = uint8,
# q = int64, f = float, 5s = 5 raw bytes. NEVER use native packing on the wire.
header = struct.pack("!I", len(payload))          # 4-byte big-endian length
struct.unpack("!I", b'\x00\x00\x01\x00')          # (256,)
struct.calcsize("!IHH")                           # 8
```

Minor details:

- Without `!` (or `>/<`), `struct` uses **native** endianness and alignment padding — `struct.pack("IH", ...)` may be 6 bytes with padding on some platforms. Always explicit byte order for wire formats.
- For text protocols prefer `\n`-delimited JSON (“JSON Lines”) — debuggable with `nc`, zero packing bugs.

8.3 Reference implementation: length-prefixed protocol

```
import socket, struct, json

_MAX = 16 * 1024 * 1024  # 16 MiB cap — always bound untrusted lengths!

def send_msg(sock: socket.socket, payload: bytes) -> None:
    sock.sendall(struct.pack("!I", len(payload)) + payload)

def _recv_exact(sock, n):
    buf = bytearray()
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed mid-message")
        buf.extend(chunk)
    return bytes(buf)

def recv_msg(sock: socket.socket) -> bytes:
    (length,) = struct.unpack("!I", _recv_exact(sock, 4))
    if length > _MAX:
        raise ValueError(f"refusing {length}-byte message")
    return _recv_exact(sock, length)

# JSON convenience layer
def send_json(sock, obj): send_msg(sock, json.dumps(obj).encode())
def recv_json(sock): return json.loads(recv_msg(sock))
```

Security notes (interviewers ask):

- **Always cap the length** before allocating — a malicious 4-byte header claiming 4 GB is a one-packet DoS.
- Validate/parse incrementally where possible.
- The header read (`_recv_exact(sock, 4)`) can itself be slow-lorised (1 byte per minute) → combine with timeouts or a max-bytes-per-connection budget.

8.4 Delimiter framing with a buffering reader

```
class LineReader:
    def __init__(self, sock, limit=65536):
        self.sock, self.buf, self.limit = sock, bytearray(), limit

    def readline(self) -> bytes:
        while True:
            if (i := self.buf.find(b"\n")) >= 0:
                line = bytes(self.buf[:i]); del self.buf[:i + 1]
                return line
            chunk = self.sock.recv(4096)
            if not chunk:
                raise ConnectionError("EOF before newline")
            self.buf += chunk
```

```

if len(self.buf) > self.limit:          # unbounded buffer = memory DoS
    raise ValueError("line too long")

```

This is essentially what `socket.makefile('r')` gives you — writing it once by hand is a rite of passage and a common interview task.

8.5 Serialization choices

Format	When
JSON / JSON Lines	default for text protocols; debuggable
pickle	Never from untrusted peers —arbitrary code execution by design
struct	tight binary control (headers, fixed records)
Protocol Buffers / MessagePack / CBOR	versioned schemas, compact, cross-language
HTTP	you probably don't need raw sockets —but you now know what's underneath

PART 9 — CONCURRENCY: SERVING MANY CLIENTS AT ONCE

A naive `accept` → `handle` → `accept` loop serves **one client at a time**. The four scaling strategies:

concurrency strategies			
1. Thread per connection (threading)	2. Process per connection (forking / multiprocessing)	3. I/O multiplex (selectors / epoll via selectors)	4. Hybrid: prefork + <code>SO_REUSEPORT</code> or <code>threads+ep</code>
simple code, ~100s of conns, GIL-bound for CPU	true parallelism, no GIL issues, heavy memory	one thread, 10k-1M conns, single-core	what nginx/unicorn do

9.1 Thread-per-connection

(See §3.10 for the code.)

- Each accepted socket gets a `threading.Thread`.
- **Pros:** dead simple, blocking code stays blocking.
- **Cons:** GIL caps CPU-bound work; each thread 8 MB default stack → thousands of threads hurt; context-switch storms.
- **Details that matter:**
 - Daemon threads or explicit `join` on shutdown.
 - Per-connection state stays in the thread — no shared mutable state without locks.
 - For graceful shutdown: keep a `threading.Event` stop flag; sockets in blocking `recv` can be woken by `shutdown()` from another thread (that raises `EOF`'s the blocked call).

– `socketserver.ThreadingTCPServer` (Part 10) packages this pattern.

9.2 Process-per-connection / prefork

```
import os, socket, signal

with socket.create_server(("0.0.0.0", 8080)) as srv:
    for _ in range(4):
        # prefork 4 workers
        pid = os.fork()
        if pid == 0:
            # child: serve forever
            while True:
                conn, addr = srv.accept()
                # all children accept from SAME socket
                with conn:
                    conn.sendall(b"hi from pid %d\n" % os.getpid())
                    # os._exit(0) on break
            # os._exit(0) on break
        # parent reaps
    os.wait()
```

- Children share the **listening** socket via fork inheritance; kernel wakes one acceptor (modern kernels avoid the old “thundering herd”).
- No GIL; crash isolation. Costs memory and IPC complexity.
- `SO_REUSEPORT` is the modern no-fork-sharing alternative: each process binds its own socket to the same port; kernel hashes new connections across them.
- Remember §3.8: child holding a connection fd keeps it alive — close listener in clients and vice versa.

9.3 `select` → `poll` → `epoll/kqueue` → `selectors`

The big idea: **one thread asks the kernel “which of these 10 000 sockets are ready?”** and only touches those. No blocking, no threads.

Mechanism	Complexity per call	Limit	Notes
<code>select.select(rlist, wlist, xlist, timeout)</code>	$O(n)$ every call	FD_SETSIZE 1024 fds	Portable; pass lists in, get ready lists out
<code>select.poll()</code>	$O(n)$	no fd limit	Register/unregister with event masks (<code>POLLIN</code> , <code>POLLOUT</code>)
<code>select.epoll()</code>	$O(\text{ready})$	huge	Linux; edge-triggered (<code>EPOLLET</code>) option = max performance, max footguns
<code>select.kqueue()</code>	$O(\text{ready})$	huge	BSD/macOS
<code>selectors.DefaultSelector</code>	best available	best available	Use this. Portable OOP wrapper: <code>epoll</code> on Linux, <code>kqueue</code> on macOS, <code>select</code> on Windows

The `selectors` module — full non-blocking echo server

```
import selectors, socket, types
```



```

sel = selectors.DefaultSelector()

def accept_wrapper(lsock):
    conn, addr = lsock.accept()
    print("accepted", addr)
    conn.setblocking(False) # MANDATORY
    data = types.SimpleNamespace(addr=addr, inb=b"", outb=b"")
    sel.register(conn, selectors.EVENT_READ | selectors.EVENT_WRITE, data=data)

def service_connection(key, mask):
    sock, data = key.fileobj, key.data
    if mask & selectors.EVENT_READ:
        try:
            recv = sock.recv(4096)
        except BlockingIOError:
            recv = None
        if recv:
            data.outb += recv # echo: buffer it
        else: # b"" → peer closed
            print("closing", data.addr)
            sel.unregister(sock)
            sock.close()
    if mask & selectors.EVENT_WRITE:
        if data.outb:
            sent = sock.send(data.outb) # partial sends happen here constantly
            data.outb = data.outb[sent:]

lsock = socket.create_server(("127.0.0.1", 65432))
lsock.setblocking(False)
sel.register(lsock, selectors.EVENT_READ, data=None) # data=None marks the listener

try:
    while True:
        for key, mask in sel.select(timeout=None): # the event loop
            if key.data is None:
                accept_wrapper(key.fileobj)
            else:
                service_connection(key, mask)
finally:
    sel.close()

```

The non-blocking playbook (memorize these rules):

1. **Every** socket registered must be `setblocking(False)`.
2. `recv` may raise `BlockingIOError` even after readability — treat as “no data”.
3. `send` is partial **all the time** — always keep an output buffer and register `EVENT_WRITE` only while it is non-empty (else the loop spins at 100% CPU).
4. `recv() == b""` → unregister **then** close. Unregistering after close raises `KeyError/ValueError`.

- 5. Errors: expect `ConnectionResetError` inside handlers; on any fatal error for a connection, unregister + close — never let one bad client kill the loop.
- 6. `DefaultSelector` on **Windows is select-based** → ~512-fd practical limit and sockets only (no pipes). For Windows scale, that's what `asyncio's ProactorEventLoop` (IOCP) solves.
- 7. Edge-triggered (`EPOLLET`) means you must drain until `EAGAIN` — the `selectors` module doesn't expose it; drop to `select.epoll` directly if you ever need it.
- 8. Waking the loop from another thread: `socketpair()` registered for read; writer thread pushes a byte.

Raw `select` (know it for interviews)

```
import select
readable, writable, exceptional = select.select([srv], [], [], 1.0)
if srv in readable:
    conn, addr = srv.accept()
```

`select` also works on pipes/files on Unix — `selectors.SelectSelector` inherits that.

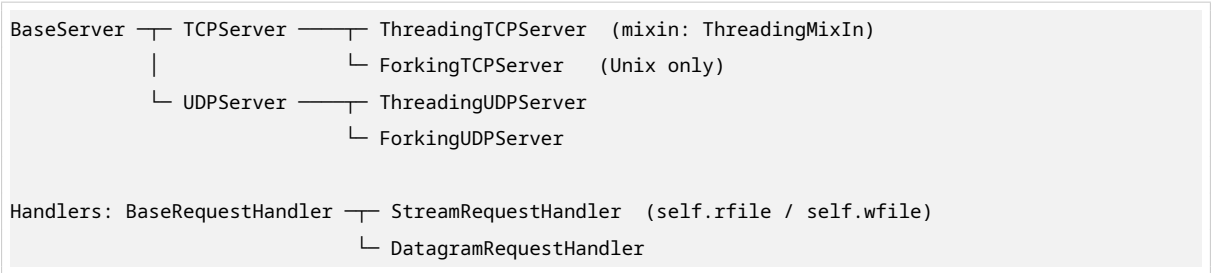
9.4 Choosing a model (decision guide)

Situation	Pick
< 200 clients, code clarity matters	Threads (or <code>ThreadingTCPServer</code>)
CPU-bound per-request work	Processes (<code>prefork/concurrent.futures.ProcessPoolExecutor</code>)
Thousands of mostly-idle connections (chat, push, proxies)	<code>selectors</code> / <code>asyncio</code>
Maximum throughput on Linux	<code>epoll</code> + edge-trigger, or <code>uvloop/asyncio</code>
Need TLS + lots of idle conns	<code>asyncio</code> or <code>selectors</code> (TLS costs per-thread stacks are fine, but <code>threads+TLS+10k conns</code> is wasteful)

PART 10 — **socketserver: THE BATTERIES-INCLUDED FRAMEWORK**

`socketserver` wraps the accept loop + dispatch. You subclass a **handler**; it does the rest.

10.1 The class map



10.2 TCP echo with `StreamRequestHandler`

```
import socketserver

class EchoHandler(socketserver.StreamRequestHandler):
    """self.request = the connection socket; self.client_address = peer."""
    def handle(self):
        self.wfile.write(b"Welcome. Lines you send come back shouted.\n")
        for line in self.rfile:          # buffered file-like reading
            self.wfile.write(line.upper()) # buffered writing (auto-flush off)

if __name__ == "__main__":
    with socketserver.ThreadingTCPServer(("127.0.0.1", 65432), EchoHandler) as server:
        server.daemon_threads = True      # don't hang exit on live clients
        server.serve_forever()            # Ctrl-C to stop
```

10.3 Handler lifecycle (override points)

```
class MyHandler(socketserver.BaseRequestHandler):
    def setup(self):    ...    # per-connection init (auth state, db handle)
    def handle(self):   ...    # the work — REQUIRED override
    def finish(self):   ...    # cleanup (runs even if handle raised)
```

10.4 UDP version

```
class UDPHandler(socketserver.BaseRequestHandler):
    def handle(self):
        data, sock = self.request          # for UDP, request = (bytes, socket)
        sock.sendto(data.upper(), self.client_address)

socketserver.ThreadingUDPServer(("127.0.0.1", 9999), UDPHandler).serve_forever()
```

10.5 Server attributes & methods you must know

Member	Meaning
<code>server_address</code> , <code>socket</code>	bound address, listener socket
<code>serve_forever(poll_interval=0.5)</code>	blocking loop (internally uses selectors!)
<code>shutdown()</code>	call from another thread —makes <code>serve_forever</code> return
<code>server_close()</code>	close the listener
<code>handle_request()</code> / <code>handle_timeout()</code>	single-step variants
<code>timeout</code>	makes <code>handle_request</code> return periodically
<code>allow_reuse_address = True</code>	class attr → sets <code>SO_REUSEADDR</code> before bind (default False on <code>TCPServer</code> !)
<code>daemon_threads</code> , <code>block_on_close</code>	threading mixin controls
<code>request_queue_size</code>	backlog (default 5 —raise it!)
<code>address_family</code>	set to <code>AF_INET6</code> for v6 servers

Minor details:

- `ThreadingTCPServer.allow_reuse_address = True` is the classic recipe line; without it, restarts hit `EADDRINUSE`.
- `serve_forever` polls at `poll_interval`; `shutdown()` takes up to that long to take effect.
- One handler instance per connection — instance attrs are per-connection state.
- For Unix domain sockets: `socketserver.UnixStreamServer` / `UnixDatagramServer`.
- It's fine for small/medium services. For high-concurrency you'll outgrow thread-per-connection — that's Part 9.3/14.

PART 11 — TLS/SSL: ENCRYPTED SOCKETS WITH `ssl`

11.1 Concept

`ssl` wraps a plain TCP socket in a TLS layer (OpenSSL under the hood). The wrapper object (`SSLSocket`) speaks `send/recv` like a normal socket but all bytes are encrypted/authenticated.

Always use an `SSLContext` — it holds certificates, verification policy, protocol versions. Never call the deprecated `ssl.wrap_socket()` directly.

11.2 TLS server

Generate a self-signed cert for dev:

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem \
-days 365 -nodes -subj "/CN=localhost"
```

```
import socket, ssl

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile="cert.pem", keyfile="key.pem")

with socket.create_server(("0.0.0.0", 8443)) as listener:
    with context.wrap_socket(listener, server_side=True) as tls_listener:
        while True:
            conn, addr = tls_listener.accept()  # handshake happens HERE (lazily on accept)
            with conn:
                print("cipher:", conn.cipher())
                conn.sendall(b"encrypted hello\n")
```

Wrapping the `listener` means every accepted connection is already an `SSLSocket`.

11.3 TLS client (the secure-by-default way)

```
import socket, ssl

context = ssl.create_default_context()  # verifies certs + hostname against system CA store
with socket.create_connection(("example.com", 443), timeout=10) as sock:
```

```

with context.wrap_socket(sock, server_hostname="example.com") as ssock: # SNI!
    ssock.sendall(b"GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n")
while chunk := ssock.recv(4096):
    print(chunk.decode(errors="replace"), end="")

```

server_hostname is mandatory when **check_hostname=True** (the default in `create_default_context`). It drives **SNI** (which cert the server presents) and **hostname verification**.

11.4 Verification knobs

Setting	Effect
<code>context.verify_mode = ssl.CERT_REQUIRED</code>	default for clients; server must present trusted cert
<code>context.check_hostname = True</code>	cert CN/SAN must match <code>server_hostname</code>
<code>context.load_verify_locations(cafile=...)</code>	custom CA bundle (corporate proxies, private CAs)
<code>context.load_cert_chain(...)</code> on client	mutual TLS (client certificates)
<code>context.minimum_version =</code> <code>ssl.TLSVersion.TLSv1_3</code>	version floor (1.2+ is the modern floor)
<code>CERT_NONE + check_hostname=False</code>	disables all verification —dev only!

11.5 Minor details that matter

- The TLS handshake is **lazily** triggered on first I/O (or `do_handshake()`); with non-blocking sockets it raises `SSLWantReadError`/`SSLWantWriteError` and must be retried via selector — `asyncio` handles this for you.
- `SSLSocket.recv()` semantics differ slightly: zero-return means clean TLS shutdown (`close_notify`); an abrupt TCP RST underneath raises `SSLEOFError`/`ConnectionResetError`.
- `ssock.getpeercert()` → client/server certificate details (auth decisions).
- `context.session_stats()`, `SSLSession` resumption for perf.
- TLS over UDP = **DTLS**, not supported by Python's `ssl`. Use QUIC libraries instead.
- Self-signed cert client-side trust: `context.load_verify_locations("cert.pem")`.

PART 12 — UNIX DOMAIN SOCKETS & EXOTIC FAMILIES

12.1 Unix domain sockets (AF_UNIX)

Same-machine IPC through the filesystem — no TCP stack, lower latency, and filesystem permissions as access control. This is how Docker (`/var/run/docker.sock`) and most databases talk locally.

```

# server
import socket, os
PATH = "/tmp/echo.sock"
if os.path.exists(PATH): os.unlink(PATH) # stale socket file blocks bind!

with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as s:
    s.bind(PATH); s.listen()
    conn, _ = s.accept()
    with conn:

```

```

        conn.sendall(conn.recv(1024))
os.unlink(PATH)                                # clean up on exit

# client
with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as s:
    s.connect(PATH)
    s.sendall(b"local hello")
    print(s.recv(1024))

```

Minor details:

- `bind` creates a real file; it is **not** removed automatically — `unlink` before `bind` and after `close`.
- Linux **abstract sockets** start with `"\0"` (`"\0myapp"`) — no filesystem entry, no cleanup.
- `socket.socketpair()` = pre-connected pair; perfect for selector wakeups and tests.
- Passing **file descriptors** between processes over Unix sockets: `sendmsg/recvmmsg` with `socket.SCM_RIGHTS` ancillary data — the mechanism behind `systemd` socket activation and `fd-passing` supervisors.
- `SO_PEERCRED` (Linux) lets the server authenticate the client's UID/PID.

12.2 Other families (know they exist)

Family	Use
<code>AF_PACKET</code> (Linux, root)	Read/write raw Ethernet frames —sniffers, custom L2
<code>AF_NETLINK</code> (Linux)	Talk to kernel (routes, interfaces, uevents)
<code>AF_BLUETOOTH</code>	RFCOMM/L2CAP
<code>AF_CAN</code>	CAN bus (automotive)
<code>AF_VSOCK</code>	VM hypervisor channels
<code>AF_ALG</code>	Kernel crypto API

PART 13 — RAW SOCKETS & PACKET CAPTURE (demystified)

```

# ICMP ping capture, Linux, needs root:
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
packet, addr = s.recvfrom(65535) # includes the IP header (unlike DGRAM sockets)

```

- `SOCK_RAW` over `AF_INET`: you receive (and can send, with `IP_HDRINCL`) full IP packets. `Root/CAP_NET_RAW` required.
- `AF_PACKET` on Linux: raw Ethernet frames, the basis of `tcpdump`-like tools:

```

s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(0x0003)) # ETH_P_ALL
s.bind(("eth0", 0))

```

```
frame = s.recv(65535)
eth_dst, eth_src, eth_proto = frame[0:6], frame[6:12], frame[12:14]
```

- **Windows:** raw TCP was removed (XP SP2); promiscuous capture uses `socket.IPPROTO_IP + ioctl(SIO_RCVALL, RCVALL_ON)`.
- Real projects: use **Scapy** — it wraps all of this with packet crafting/parsing. Raw sockets are for understanding, not for building parsers by hand (unless that's the exercise — see Part 18).

PART 14 — asyncio NETWORKING (THE MODERN PATH)

You already understand the hard part (non-blocking sockets + event loop = Part 9.3). `asyncio` is that pattern with coroutine ergonomics. Two API levels:

14.1 High-level streams (start here)

```
import asyncio

async def handle(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    addr = writer.get_extra_info("peername")
    while data := await reader.read(4096):      # b"" on EOF — same rule!
        writer.write(data.upper())
        await writer.drain()                   # backpressure! waits if peer is slow
    writer.close()
    await writer.wait_closed()

async def main():
    server = await asyncio.start_server(handle, "127.0.0.1", 65432)
    async with server:
        await server.serve_forever()

asyncio.run(main())
```

```
# client
async def client():
    r, w = await asyncio.open_connection("127.0.0.1", 65432)
    w.write(b"ping\n"); await w.drain()
    print(await r.readline())
    w.close(); await w.wait_closed()
```

14.2 UDP & low-level

```
loop = asyncio.get_running_loop()
transport, protocol = await loop.create_datagram_endpoint(MyDatagramProtocol, local_addr=("0.0.0.0",
↪ 9999))
# or wrap an EXISTING socket: loop.create_connection(sock=raw_sock) / create_datagram_endpoint(sock=...)
```

14.3 Key details

- `await writer.drain()` = flow control; skipping it lets memory explode on fast-producer/slow-consumer.
- `reader.readexactly(n)` is `recv_exact` built-in; `readuntil(b"\n")` with `LimitOverrunError`; the default stream limit is 64 KiB (`limit=` param).
- Timeouts: `asyncio.wait_for(coro, 5)`.
- TLS: `ssl=` argument to `start_server/open_connection` — `asyncio` manages the non-blocking handshake dance.
- Under the hood on Linux: `epoll`; macOS: `kqueue`; Windows: **ProactorEventLoop (IOCP)** — which is why `asyncio` is the answer to “selectors doesn’t scale on Windows”.
- You can always drop to your own socket: pass `sock=` to `loop.create_connection`, or use `loop.sock_recv/sock_send/sock_accept/sock_connect` — coroutine wrappers around your socket object.

PART 15 — ERRORS & EXCEPTIONS: THE COMPLETE MAP

15.1 Exception hierarchy

```
OSError (≡ socket.error since 3.3)
├─ BlockingIOError      EAGAIN/EWOULDBLOCK — non-blocking op can't proceed now
├─ InterruptedError     EINTR — signal hit (auto-retried since 3.5, PEP 475)
├─ TimeoutError         ETIMEDOUT — kernel timeout; socket.timeout is an ALIAS of this
├─ ConnectionError
│   ├─ BrokenPipeError   EPIPE — wrote to a peer that closed (SIGPIPE converted)
│   ├─ ConnectionAbortedError ECONNABORTED — e.g. accept queue entry died
│   └─ ConnectionRefusedError ECONNREFUSED — RST: nothing listening (TCP) / ICMP port unreachable
│       ↪ (UDP)
│   └─ ConnectionResetError ECONNRESET — peer aborted (RST): crashed app, SO_LINGER(1,0), ...
├─ socket.gaierror      getaddrinfo/DNS failures (errno like EAI_NONAME)
└─ socket.herror        legacy gethostby* failures

ssl.SSLError (subclass of OSError)
├─ ssl.SSLCertVerificationError — bad/self-signed/expired/hostname-mismatch certs
└─ ssl.SSLEOFError            — connection cut without TLS close_notify
```

Catch `OSError` for the everything-bucket; catch the specific subclasses when recovery differs (retry on `ConnectionRefusedError`, drop client on `ConnectionResetError`...).

15.2 errno cheat sheet

errno	# (Linux)	Typical cause
ECONNREFUSED	111	connect to closed port
ECONNRESET	104	peer sent RST
EPIPE	32	send after peer closed
ETIMEDOUT	110	SYN/keepalive timed out

Table 22 – continued

errno	# (Linux)	Typical cause
EADDRINUSE	98	bind on busy/TIME_WAIT port
EADDRNOTAVAIL	99	bind non-local IP / ephemeral exhaustion on connect
EACCES	13	privileged port / broadcast without option / raw without root
EINPROGRESS	115	non-blocking connect started
EAGAIN/EWOULDBLOCK	11	non-blocking I/O would block
EHOSTUNREACH	113	ICMP host unreachable
EMFILE/ENFILE	24/23	fd limit hit (raise ulimit -n)

```
import errno
try:
    s.connect(addr)
except OSError as e:
    if e.errno == errno.ECONNREFUSED:
        ...          # e.strerror gives human text; str(e) is '[Errno 111] Connection refused'
```

15.3 The “what does this crash mean” quick decoder

- **ConnectionResetError on first recv after accept** → client disconnected between handshake and accept; ignore and continue.
- **BrokenPipeError on send** → the peer closed earlier; your earlier sends went into the void silently. TCP reports loss *lazily*.
- **socket.timeout on accept** → your listener has a timeout set; loop and re-accept (common in graceful-shutdown patterns).
- **OSError: [Errno 24] Too many open files** → fd exhaustion: leaking sockets, or need higher ulimit -n, or too many concurrent conns.
- **gaierror: [Errno -2]** → DNS name doesn’t resolve (typo, no network, missing /etc/hosts entry).
- **SSLEOError** → peer cut TCP without TLS goodbye — often a non-TLS client hitting your TLS port (or a port scan).

PART 16 — DEBUGGING & TESTING TOOLKIT

Tool	Use
nc -l 127.0.0.1 9000 / nc 127.0.0.1 9000	Instant throwaway server/client
nc -u -l ...	UDP variant
telnet host port	Interactive TCP poking
ss -tlnp / netstat -tlnp	Who’s listening; state counts (TIME_WAIT!)
tcpdump -i lo port 65432 -X	See the actual bytes
Wireshark	GUI deep-dive, follow TCP stream
strace -f -e trace=network python app.py	See every syscall with args
lsof -i :65432	Which process owns the port

Table 23 – continued

Tool	Use
<code>openssl s_client -connect host:443</code>	TLS handshake debugging
<code>curl -v telnet://host:port</code>	Another poke tool
<code>python -m http.server</code>	A known-good peer in one command

Testing discipline:

- Bind port **0** in tests; read the port from `getsockname()` — no conflicts, parallel-test safe.
- Test over **127.0.0.1**, but remember loopback hides framing bugs — add a test that drip-feeds bytes (send 1 byte at a time) to prove your framing.
- `socket.socketpair()` for unit-testing protocol functions without any network.
- Assert on **bytes on the wire**, not just return values.

PART 17 — THE PITFALLS CHECKLIST (print this)

1. Assuming `send()` sent everything → use `sendall()` or check the return.
2. Assuming one `recv()` = one message → implement framing (Part 8).
3. Treating `recv() == b""` as “try again later” → it’s EOF; close.
4. Forgetting `SO_REUSEADDR` → mysterious `EADDRINUSE` on restart.
5. Backlog too small under burst load → raise it; but fix accept-loop speed too.
6. Blocking `recv` on an idle connection forever → heartbeats or TCP keepalive.
7. No timeout anywhere → one dead peer hangs your whole thread pool.
8. Mixing `makefile()` reads with raw `recv()` on the same socket → lost buffered bytes.
9. Timeout + `makefile()` reads → docs warn: unreliable.
10. Forgetting `SO_BROADCAST` → `PermissionError` on send.
11. UDP buffer smaller than datagrams → silent truncation.
12. Native `struct` packing on the wire → endianness/alignment corruption; use `!`.
13. No cap on length-prefixed sizes → one header = memory DoS.
14. Threads sharing one socket for concurrent `send` → interleaved garbage; serialize sends with a lock (or one writer thread + queue).
15. Non-blocking loop registering `EVENT_WRITE` permanently → 100% CPU spin.
16. Catching only `socket.error` on Python 3 code that also needs `TimeoutError` → they merged into `OSError`; catch `OSError`.
17. `close()` without `shutdown(SHUT_WR)` when the peer reads-until-EOF → peer hangs.
18. Leaking sockets (no `with`, no `close` in `finally`) → fd exhaustion at scale.
19. Assuming `gethostbyname(gethostname())` gives your LAN IP → use the UDP-connect trick.
20. Using `pickle` on untrusted input → RCE. Use JSON/protobuf.
21. `verify_mode = CERT_NONE` in production → you’ve disabled TLS’s point.
22. Forgetting Windows selector limits (sockets only, ~512 fds) → `asyncio Proactor` instead.
23. Killing `TIME_WAIT` with `SO_LINGER(1,0)` → data corruption risk; fix your protocol instead.
24. Ignoring backpressure in `asyncio` (`drain()`) → unbounded memory growth.

PART 18 — CAPSTONE PROJECTS (BUILD ALL FIVE)

18.1 Multi-client chat server (threads + framing)

Requirements: nicknames, /quit, broadcast to all, clean disconnects, length-prefixed JSON messages.

Teaches: shared-state locks, framing, per-client writer queues.

```
import socket, threading, json, struct

lock = threading.Lock()
clients = set() # sockets

def send_msg(sock, obj):
    data = json.dumps(obj).encode()
    sock.sendall(struct.pack("!I", len(data)) + data)

def broadcast(obj, exclude=None):
    data = json.dumps(obj).encode()
    frame = struct.pack("!I", len(data)) + data
    with lock:
        dead = []
        for c in clients:
            if c is exclude: continue
            try: c.sendall(frame)
            except OSError: dead.append(c)
        for c in dead: clients.discard(c)

def handle(conn, addr):
    with conn:
        nick = f"{addr[0]}:{addr[1]}"
        broadcast({"type": "join", "who": nick})
        try:
            while True:
                hdr = conn.recv(4)
                if not hdr: break
                (n,) = struct.unpack("!I", hdr)
                body = b""
                while len(body) < n:
                    chunk = conn.recv(n - len(body))
                    if not chunk: break
                    body += chunk
                msg = json.loads(body)
                if msg.get("text") == "/quit": break
                broadcast({"type": "chat", "who": nick, "text": msg["text"]}, exclude=conn)
            except (ConnectionError, OSError, json.JSONDecodeError):
                pass
        finally:
            with lock: clients.discard(conn)
            broadcast({"type": "leave", "who": nick})

with socket.create_server(("0.0.0.0", 7000)) as srv:
    while True:
```

```

conn, addr = srv.accept()
with lock: clients.add(conn)
threading.Thread(target=handle, args=(conn, addr), daemon=True).start()

```

18.2 Port scanner (connect_ex + timeouts + threads)

```

import socket
from concurrent.futures import ThreadPoolExecutor

def scan(host, port, timeout=0.5):
    with socket.socket() as s:
        s.settimeout(timeout)
        return port if s.connect_ex((host, port)) == 0 else None # 0 == open

target = "127.0.0.1"
with ThreadPoolExecutor(max_workers=200) as ex:
    open_ports = [p for p in ex.map(lambda p: scan(target, p), range(1, 1025)) if p]
print("open:", open_ports)

```

Upgrade path: banner grabbing (recv after connect), getservbyport labels, SYN-scan discussion (raw sockets / scapy), rate-limiting ethics note — **only scan machines you own.**

18.3 File transfer (TCP + framing + progress + integrity)

Protocol: !I filename length, filename UTF-8, !Q file size, then raw bytes; SHA-256 trailer; server verifies. *Teaches:* streaming without loading files into memory, recv_exact, hashing on the fly.

```

# sender core
import socket, struct, hashlib, os

def send_file(sock, path):
    name = os.path.basename(path).encode()
    size = os.path.getsize(path)
    sock.sendall(struct.pack("!I", len(name)) + name + struct.pack("!Q", size))
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while chunk := f.read(65536):
            sock.sendall(chunk); h.update(chunk)
    sock.sendall(h.digest()) # 32-byte trailer

```

Upgrade: sock.sendfile(f) zero-copy version; resume-support via offset negotiation.

18.4 selectors-based proxy (the rite of passage)

TCP forwarding proxy: accept → connect upstream → shuttle bytes both ways with per-connection buffers, handling half-closes (SHUT_WR propagation) and backpressure (stop reading a side whose peer's buffer is full). *Teaches:* everything in Part 9 at once. Target: < 150 lines.

18.5 DNS-over-UDP mini-resolver (UDP + struct parsing)

Send a hand-built DNS query for `example.com` A to `8.8.8.8:53`, parse the answer section's A record with `struct`. *Teaches*: real binary protocol work, UDP timeouts/retries, header bit-fields (`struct.pack("!HHHHH", id, flags, 1, 0, 0, 0)` + QNAME encoding).

PART 19 — INTERVIEW & SELF-TEST QUESTIONS

Conceptual

1. Why can one TCP port serve a million connections simultaneously? (*5-tuple identity*.)
 2. What exactly does `accept()` return, and why is the listening socket never used for data?
 3. Explain both queues behind `listen(backlog)`. What happens when they fill?
 4. Your server restarts and crashes with `EADDRINUSE`. Why, and the fix? (*TIME_WAIT; SO_REUSEADDR*.) Why doesn't it happen for the *client* side? (*Clients use fresh ephemeral ports*.)
 5. Difference between `shutdown()` and `close()`? When does a forked child matter?
 6. TCP guarantee: does `sendall()` success mean the peer received the data? (*No — kernel accepted it. Failures surface later*.)
 7. Why does `recv` need a loop? Write `recv_exact` from memory.
 8. What is Nagle's algorithm and when do you kill it? (*Small-write latency; TCP_NODELAY for RPC*.)
 9. Blocking vs non-blocking connect — how do you learn the result of the latter? (*select-for-writable + getsockopt(SO_ERROR)*.)
 10. Why is UDP's `connect()` not a handshake? What three things does it do?
 11. Compare `select/poll/epoll` complexity. Why is `epoll` $O(\text{ready})$?
 12. What is the C10k problem and what solved it? (*Thread-per-conn didn't scale; event-driven I/O*.)
- Practical** 13. Write an echo server that never blocks on a single client. (*selectors server from §9.3*.) 14. Frame a protocol to send variable-length messages. Then: how do you stop a 4 GB allocation attack? 15. Detect a dead peer on an idle TCP connection. (*Heartbeats; SO_KEEPALIVE + tuning*.) 16. Why might `makefile + settimeout` corrupt your protocol? 17. Server handles 10k mostly-idle TLS connections — threads or selectors? Justify. 18. How does `create_server` differ from raw `socket+bind+listen`? (*REUSEADDR done per-platform, dual-stack option, backlog default*.) 19. Explain `SO_REUSEADDR` vs `SO_REUSEPORT` with use cases. 20. Trace the bytes: client `sendall(b"AB")`, `sendall(b"CD")`; server does `recv(3)` twice. What can it get? (*Any split: b"ABC"+b"D", b"A"+b"BCD", ... — never an error, always just stream*.)
-

PART 20 — ONE-PAGE CHEAT SHEET

```
import socket, struct, selectors, threading, ssl
```

```

# — CREATE —————
s = socket.socket()                                # AF_INET + SOCK_STREAM (TCP)
u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # UDP
s6 = socket.socket(socket.AF_INET6, socket.SOCK_STREAM)
ux = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)

# — TCP SERVER —————
srv = socket.create_server(("0.0.0.0", 8080))        # bind+listen+REUSEADDR (3.8+)
conn, addr = srv.accept()
data = conn.recv(4096)                             # b"" → peer closed
conn.sendall(b"reply")
conn.shutdown(socket.SHUT_WR); conn.close()

# — TCP CLIENT —————
c = socket.create_connection(("example.com", 80), timeout=5)
c.getpeername(); c.getsockname(); c.settimeout(10)

# — UDP —————
u.bind(("", 9999)); data, addr = u.recvfrom(65535); u.sendto(b"ok", addr)
u.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1) # before broadcasting

# — OPTIONS —————
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) # before bind
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1) # kill Nagle
s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
s.getsockopt(socket.SOL_SOCKET, socket.SO_ERROR)        # nb-connect result

# — MODES —————
s.setblocking(False) # → BlockingIOError on would-block
s.settimeout(5)      # → socket.timeout/TimeoutError

# — DNS / ADDRESSES —————
socket.getaddrinfo("host", 443, 0, socket.SOCK_STREAM) # iterate the results!
socket.inet_pton(socket.AF_INET, "1.2.3.4")
socket.htons(80); struct.pack("!I", n)                 # network byte order

# — FRAMING —————
sock.sendall(struct.pack("!I", len(msg)) + msg)         # length-prefix send
# recv: loop until exact N bytes; cap N!

# — CONCURRENCY —————
threading.Thread(target=handle, args=(conn,), daemon=True).start()
sel = selectors.DefaultSelector(); sel.register(sock, selectors.EVENT_READ, cb)
for key, mask in sel.select(): key.data(key.fileobj, mask)

# — TLS —————
ctx = ssl.create_default_context()                      # client: verify on
ss = ctx.wrap_socket(raw, server_hostname="example.com")
srv_ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
srv_ctx.load_cert_chain("cert.pem", "key.pem")

```

```
# — CATCH —————  
except OSError as e:      # covers timeout, reset, refused, gaierror...  
    e.errno, e.strerror
```

GLOSSARY (60 seconds)

- **5-tuple** — protocol + local addr/port + remote addr/port = unique connection ID.
- **Backlog** — max pending connections queued for `accept()`.
- **Ephemeral port** — short-lived client-side port (49152–65535).
- **FIN / RST** — graceful close / abortive reset segment.
- **Framing** — how a byte stream is cut back into messages.
- **Half-close** — `shutdown(SHUT_WR)`: done sending, still reading.
- **Keepalive** — probes to detect dead idle peers (kernel or app level).
- **MTU / MSS** — max frame size on a link / max TCP payload per segment.
- **Nagle** — TCP small-packet batching algorithm (disable with `TCP_NODELAY`).
- **SNI** — TLS extension naming the host you want, enabling virtual-hosted certs.
- **TIME_WAIT** — 60 s graveyard state after active close; why servers need `SO_REUSEADDR`.
- **Zero-copy** — `sendfile`: data moves file→NIC without user-space copies.

End of notes. Rule of thumb for mastery: if you can rebuild Parts 3, 8, and 9.3 from a blank file and explain every line, you are done.